BACHELOR'S THESIS (TFG) · UC3M

# One flight, four papers

Script for a 5-minute presentation (8 slides)

[Your name] · Supervisor: [name] · September 2026

## The thread

The talk puts reinforcement learning first. After every uploaded flight, an RL bandit (TAILOR) decides which tiles a person labels. That comes right after the intro, followed by how it compares with the previous thesis's "one frame every 3 seconds". The other three papers are then presented as **what makes the RL loop work in practice**: how to use the labels, how to retrain cheaply, and how to compare flights in different weather. Every paper slide shows the four steps at the top, with RL first and the current step highlighted.

| # | Step | Paper | Time |
|---|---|---|---|
| 1 | Title: RL as the main idea | — | 0:15 |
| 2 | RL decides what to label; three supporting questions | — | 0:30 |
| 3 | **1 · Choose tiles (RL)** | **TAILOR (NeurIPS 2023)** | **1:10** |
| 4 | **Bandit vs. every 3 s** | **— (previous thesis)** | **0:45** |
| 5 | 2 · Use the labels | Simple Supervision (arXiv 2026) | 0:40 |
| 6 | 3 · Retrain | On Warm-Starting (NeurIPS 2020) | 0:35 |
| 7 | 4 · Compare flights | LUF-net (Plant Phenomics 2026) | 0:30 |
| 8 | The shared lesson + questions | — | 0:35 |

About 750 words: 5:00 at a brisk pace (150 words per minute). The RL part (slides 3–4) takes almost two minutes. If running late, shorten slide 7.

## Script

### Slide 1 · One flight, four papers

**0:00 – 0:15**

On screen: title slide

> Hi. My TFG extends CARLA, the project that detects cardilla in drone video. The main idea is to use reinforcement learning so the detector keeps learning from every new flight, with as little labeling as possible.

## Slide 2 · RL decides what to label

**0:15 – 0:45**

On screen: the RL decision (TAILOR) on the left; three supporting questions on the right

> Today CARLA's model was trained once, on flights from a single day. But new flights keep arriving. So after every upload, a person labels a few tiles and the model is updated.
>
> **The key decision in each round is which tiles to label, and that's what reinforcement learning learns.** Around it, three practical questions remain: how to use those labels, how to retrain without making the model worse, and how to compare flights in different weather. One paper for each.

## Slide 3 · Choose tiles — TAILOR (RL)

**0:45 – 1:55**

On screen: 9 of 10 and = random

> First, the reinforcement learning: which tiles should a person label? There are many strategies, like least confident or most different, and the best one changes from dataset to dataset.
>
> TAILOR, NeurIPS 2023, doesn't pick one. Each strategy is an arm of a multi-armed bandit. For every slot in the batch, Thompson sampling picks a strategy, the person labels, and the strategy is rewarded straight from those labels: examples of rare classes are worth more. So no retraining is needed to get a reward.
>
> On ten imbalanced datasets it matched the best single strategy on nine, and an older bandit rewarded by accuracy did no better than random.
>
> It was only tested on classification, so for CARLA I redefine the reward: a tile is valuable if the detector missed a plant or made a false positive, especially in a light and altitude condition we rarely see. Each uploaded flight is one round of the bandit.

## Slide 4 · Bandit vs. every 3 seconds

**1:55 – 2:40**

On screen: two columns, "Before: every 3 s" and "Now: the bandit"

> How does this compare with what CARLA did before? The previous thesis labeled one frame every three seconds, whatever it contained. That rule ignores the detector: neighbouring frames overlap a lot, many tiles are just empty grass, and rare cases like missed plants are found only by chance.
>
> **With the bandit, every-three-seconds doesn't disappear. It becomes one of the arms.** If it really is the best strategy, the bandit learns to pick it; if not, labels move to the tiles that teach the detector most.
>
> The price is one retrain per flight, and some labels spent exploring. And every-three-seconds keeps one job: because it ignores the model, it gives an unbiased test set.
>
> **To make all this work in practice, three questions remain.**

## Slide 5 · Use the labels — Simple Supervision

**2:40 – 3:20**

On screen: +1.7 to +18.3 and −13.6

> Second: the bandit has chosen and the person has labeled. How do we use a few labels plus a whole unlabeled flight? A 2026 preprint on detectors adapting to fog and new cities says: keep it simple.
>
> RTSM: keep training on pseudo-labels, and add the real labels as ordinary examples. That improved results by 1.7 to 18.3 AP50 in every setting. Ten cleverer methods that used the labels to fix the pseudo-labels didn't reliably help, and one lost 13.6 points.
>
> **The authors chose the labeled images at random on purpose; choosing them was out of scope, which is exactly what the bandit adds.** So this gives our training recipe, and the baseline the bandit must beat: random tiles, same recipe.

### Slide 6 · Retrain — On Warm-Starting

**3:20 – 3:55**

On screen: 56.2 ⟶ 51.7% and “same accuracy”

> Third: retraining happens after every flight, so it must be cheap, and it’s tempting to just continue from the last weights.
>
> Ash and Adams, NeurIPS 2020, show that this warm start generalizes worse: on CIFAR-10, 56 versus 52 percent.
>
> Their fix is Shrink and Perturb: multiply the weights by about 0.6 and add a little noise before each round. It matched training from scratch, and was much faster.
>
> Caveat: they used no data augmentation and YOLO does, so I’ll measure it.

### Slide 7 · Compare flights — LUF-net

**3:55 – 4:25**

On screen: 9.3% ⟶ 3.9% and 4.95%

> Last: flights happen in sun and in cloud, and the bandit’s reward needs to know which.
>
> LUF-net, from Plant Phenomics 2026, corrects colour with a small network that also receives the camera settings and a light-sensor reading. Colour error drops from 9.3 percent using the image alone to 3.9.
>
> With only the exposure settings DJI already records, it’s still 5 percent.
>
> **And the same camera data labels every tile with its altitude and light condition, which is exactly what the bandit’s reward uses.** Limits: it measures colour, not detection, and training needs a colour chart.

### Slide 8 · Simple signals win

**4:25 – 5:00**

On screen: three lesson cards and two questions

> To close: the four papers share one lesson: simple signals win. A label-based reward beats accuracy, plain labels beat pseudo-label tricks, and a rescaling beats tuning. So I’ll always report the simple baselines next to the RL.
>
> Two questions for you: is a bandit enough RL for the thesis, and can I fully label one or two flights to simulate the loop? Thank you.

## Linking sentences

These lines tie every paper back to the RL; don’t cut them if you shorten the talk.

- **Slide 2:** “The key decision in each round is which tiles to label, and that’s what reinforcement learning learns.”
- **Slide 4:** “Every-three-seconds doesn’t disappear. It becomes one of the arms.” (The new method contains the old one.)
- **Slide 4 ⟶ 5:** “To make all this work in practice, three questions remain.”
- **Slide 5:** “Choosing them was out of scope, which is exactly what the bandit adds.” (Simple Supervision and TAILOR answer complementary questions.)
- **Slide 7:** “The same camera data labels every tile with its altitude and light condition, which is exactly what the bandit’s reward uses.” (LUF-net feeds the RL reward.)
- **Slide 8:** “Simple signals win.” (The lesson three of the papers share.)

## Backup answers for questions

| If asked… | Answer |
|---|---|
| Will the bandit really beat every 3 s? | Not guaranteed. TAILOR matched the best single strategy on 9 of 10 datasets, but it had far more labels per round. The fair test: same number of labeled tiles for each method, same training recipe, and compare mAP50 on held-out flights against the number of labels. |
| Why keep every 3 s at all? | Because it ignores the model, it is an unbiased sample of real flights. A test set chosen by the bandit would be full of hard cases and would make every model look worse than it is in the field. |
| Why a bandit and not a "real" RL policy? | A learned policy (e.g. MGRAL, ICASSP 2026, policy gradient with an mAP reward) needs many retrains per update. With one GPU and one round per flight, a bandit is what fits. A learned policy is a possible extension. |
| Why not use the mAP gain as the reward? | It needs a retrain per reward and is noisy between seeds. In TAILOR, the accuracy-rewarded bandit (ALBL) did no better than random. mAP is kept for evaluation, and comparing both rewards is itself an experiment. |
| TAILOR has only been tested on classification. Why should it work for detection? | It isn't known yet; that is the gap the thesis fills. Its multi-label form fits once each tile is described by yes/no attributes (contains cardilla, missed plant, false positive, condition). |
| What if the bandit doesn't beat random? | That is still a result: RTSM with random tiles is the baseline, and Simple Supervision shows it is strong. Reporting it honestly is part of the contribution. |
| Does LUF-net work on compressed video? | Untested; the paper used still images. It will be compared against plain colour augmentation, which is the cheap baseline. |
| Does Shrink & Perturb still matter with YOLO's augmentation? | Unknown; the paper had no augmentation. I'll compare warm start, Shrink & Perturb and restarting from COCO weights, in accuracy and GPU time. |

## Delivery tips

- Slow down on slides 3 and 4: they are the RL part and the main focus of the talk.
- On slide 4, point at the left column, then the right: the contrast is the message.
- On slides 5–7, point at the highlighted step at the top and say how it serves the bandit. That keeps the RL at the centre.
- On each paper slide, say only the two numbers on the right; the bullets are there for reading later.